



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 17/09/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para el primer ejercicio debo realizar el modelo de un cohete espacial utilizando todas las figuras proporcionadas por el profesor, para lo cual, decidí usar 3 cilindros para el cuerpo del cohete y sus propulsores, 8 conos para la nariz, ventanas y la parte de abajo de los cilindros, 3 pirámides cuadrangulares para representar el fuego que impulsa el cohete, 4 pirámides triangulares para los alerones y un cubo que une el cilindro central y el cono inferior.

```
//Cohete
// Cono azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 1.6f, 0.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.8f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
```

```
// Cilindro Gris
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 2.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 0.6f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

```
// Cono azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.6f, 0.9f, 0.0f));
model = glm::scale(model, glm::vec3(0.125f, 0.3f, 0.125f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
```

```
// Cono azul
model = baseModel;
model = glm::translate(model, glm::vec3(-0.6f, 0.9f, 0.0f));
model = glm::scale(model, glm::vec3(0.125f, 0.3f, 0.125f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
```

```
// Cilindro Gris
model = baseModel;
model = glm::translate(model, glm::vec3(0.6f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.5f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 0.6f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

```
// Cilindro Gris
model = baseModel;
model = glm::translate(model, glm::vec3(-0.6f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 1.5f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 0.6f); // Gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

```
// Pirámide triangular roja
model = baseModel;
model = glm::translate(model, glm::vec3(0.3f, -0.5f, 0.3f));
model = glm::scale(model, glm::vec3(0.2f, 0.6f, 0.2f));
model = glm::rotate(model, 120.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// Pirámide triangular roja
model = baseModel;
model = glm::translate(model, glm::vec3(-0.3f, -0.5f, 0.3f));
model = glm::scale(model, glm::vec3(0.2f, 0.6f, 0.2f));
model = glm::rotate(model, -220.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// Pirámide triangular roja
model = baseModel;
model = glm::translate(model, glm::vec3(-0.3f, -0.5f, -0.3f));
model = glm::scale(model, glm::vec3(0.2f, 0.6f, 0.2f));
model = glm::rotate(model, 300.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// Pirámide triangular roja
model = baseModel;
model = glm::translate(model, glm::vec3(0.3f, -0.5f, -0.3f));
model = glm::scale(model, glm::vec3(0.2f, 0.6f, 0.2f));
model = glm::rotate(model, -30.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMeshGeometry();
```

```
// Cubo azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, -0.85f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.1f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMeshGeometry();
```

```
// Cono gris
model = baseModel;
model = glm::translate(model, glm::vec3(-0.6f, -0.78f, 0.0f));
model = glm::scale(model, glm::vec3(0.12f, 0.03f, 0.12f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
```

```
// Cono gris
model = baseModel;
model = glm::translate(model, glm::vec3(0.6f, -0.78f, 0.0f));
model = glm::scale(model, glm::vec3(0.12f, 0.03f, 0.12f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]->RenderMeshGeometry();
```

```
// Cono azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.4f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.08f, 0.01f, 0.08f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]-->RenderMeshGeometry();
```

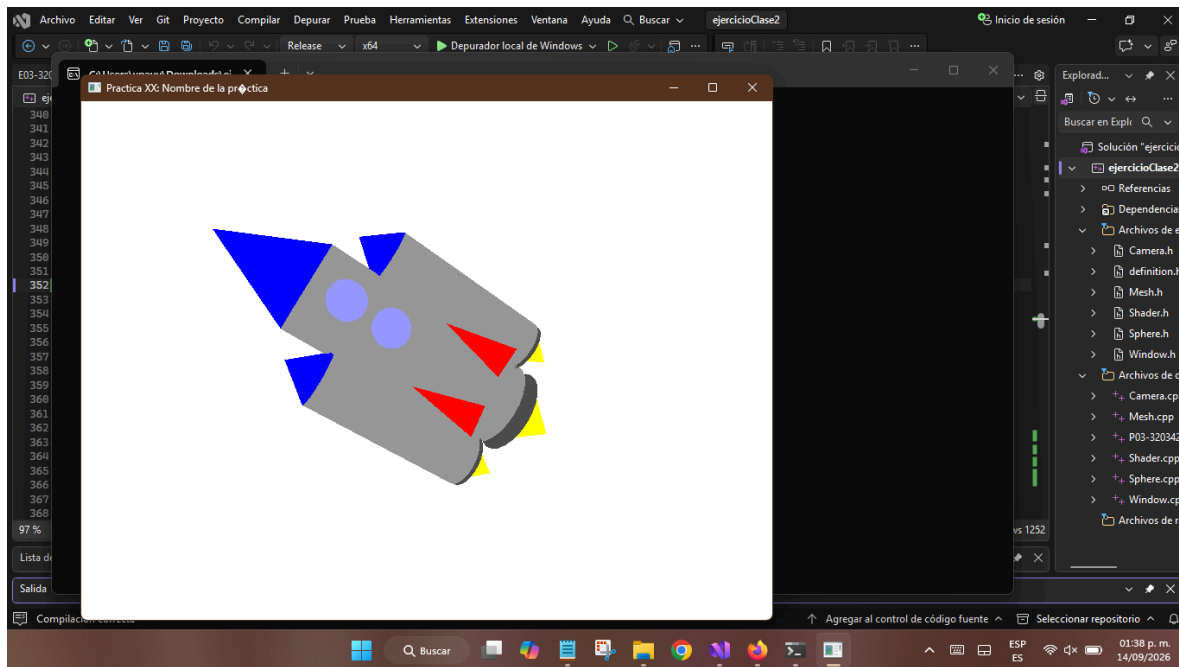
```
// Cono azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 0.9f, 0.4f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.08f, 0.01f, 0.08f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.6f, 0.6f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[3]-->RenderMeshGeometry();
```

```
// Piramide cuadrangular amarillo
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, -1.2f, 0.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]-->RenderMeshGeometry();
```

```
// Piramide cuadrangular amarillo
model = baseModel;
model = glm::translate(model, glm::vec3(-0.6f, -0.95f, 0.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]-->RenderMeshGeometry();
```

```
// Piramide cuadrangular amarillo
model = baseModel;
model = glm::translate(model, glm::vec3(0.6f, -0.95f, 0.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]-->RenderMeshGeometry();
```

La ejecución se muestra a continuación:



Para la segunda actividad se debía crear una figura con 8 pirámides como un artefacto de la película de Ender's Game, para esto, los colores de las pirámides que apuntan al exterior debían ser de 4 colores diferentes y los 4 interiores debían ser azules para formar un cuadrado a la vista. Se utilizaron únicamente pirámides cuadrangulares.

```
// 8 Piramides
// Magenta
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 0.91f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 1.0f); // Magenta
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, 0.39f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Verde
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, -0.61f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f); // Verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

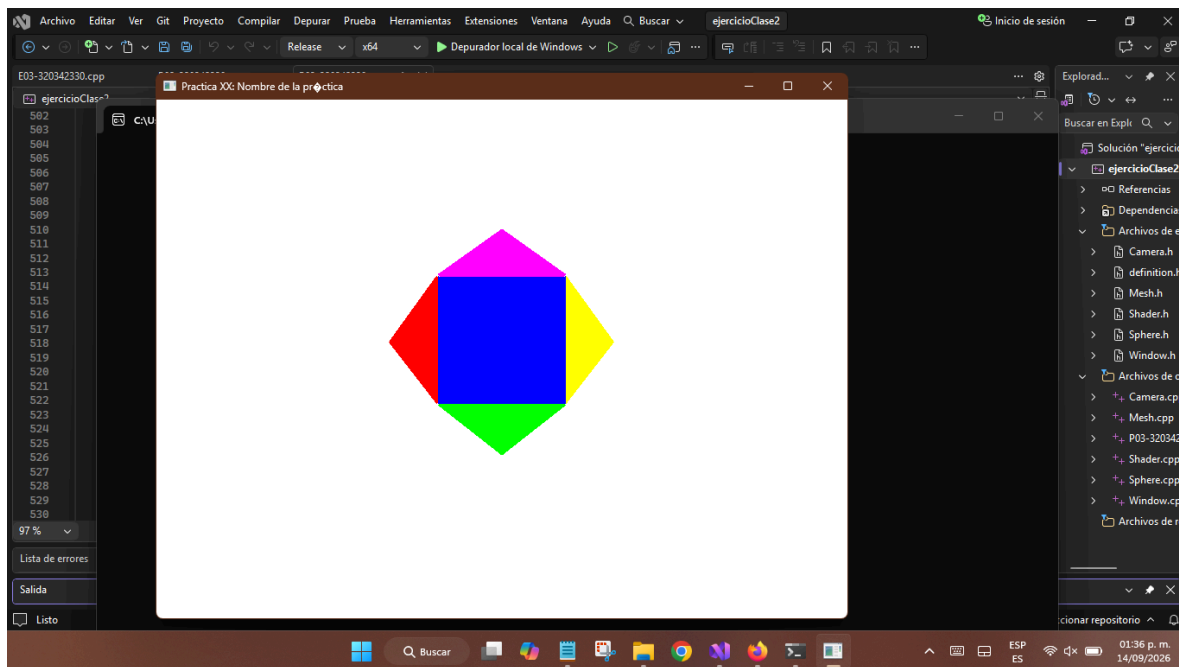
```
// Amarillo
model = baseModel;
model = glm::translate(model, glm::vec3(0.751f, 0.15f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 1.0f));
model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Azul
model = baseModel;
model = glm::translate(model, glm::vec3(0.25f, 0.15f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 1.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Rojo
model = baseModel;
model = glm::translate(model, glm::vec3(-0.76f, 0.15f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 1.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

```
// Azul
model = baseModel;
model = glm::translate(model, glm::vec3(-0.25f, 0.15f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 1.0f));
model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 1.0f); // Azul
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();
```

La ejecución final se muestra a continuación:



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Tuve problemas con la creación de la segunda figura pues no entendía muy bien cómo realizarla con las figuras dadas y no comprendía las instrucciones de colores pero no tuve ningún problema con la ejecución.

3. Conclusión

Me fue muy interesante desarrollar esta práctica aplicando las diferentes figuras proporcionadas, entendiendo de mejor manera su uso para evitar que hubiera fallos en la proyección que afectasen la vista de todas las caras de la figura. Considero que fue buena la explicación de las actividades para la clase y el

ejercicio de clase me hizo entender al completo el uso de las figuras para poder desarrollar esta práctica.